

Deep Learning II

Digitization and Data Analytics — Lecture 9

TU Dresden · MS Computational Science / Computer Science

Comprehensive Study Guide

In-depth notes · Visual diagrams · Worked examples · Exam-style Q&A; · Research-oriented questions

Covers: The power of DNNs · Convolutional Neural Networks · Recurrent Neural Networks · Self-Attention & Transformers (BERT / GPT / T5)

Table of Contents

- 1. The Power of Deep Neural Networks
- 2. Images and Convolutional Neural Networks (CNNs)
- 3. Sequences and Recurrent Neural Networks (RNNs)
- 4. Text Data, Self-Attention and Transformers
- 5. Cheat-Sheet: Key Formulas
- 6. Exam-Style Q&A; (TU Dresden format)
- 7. Calculation / Worked-Example Questions
- 8. Research-Oriented Q&A; (beyond the slides)
- 9. Likely Past-Exam-Style Questions & Model Answers
- 10. Final Takeaways & Further Reading

1. The Power of Deep Neural Networks

Classical pattern-recognition pipelines (speech recognition, image classification, ...) were built from **hand-crafted** layers of abstraction: engineers designed feature extractors (e.g. sound features, phoneme features, gradients, histograms of oriented gradients, SIFT, bag-of-visual-words) and only the final classifier stage was trainable. This required domain expertise and did not scale well to new domains or unseen variability in the data.

A **Deep Neural Network (DNN)** replaces this hierarchy with stacked trainable layers. Given enough labelled data and compute, the network learns the entire feature hierarchy end-to-end via gradient descent / backpropagation — low-level layers learn edges/phonemes, middle layers learn parts/words, and high-level layers learn objects/semantics, without any layer being manually specified.

Classical pipeline: hand-crafted feature hierarchy



Deep Neural Network: hierarchy learned automatically from data

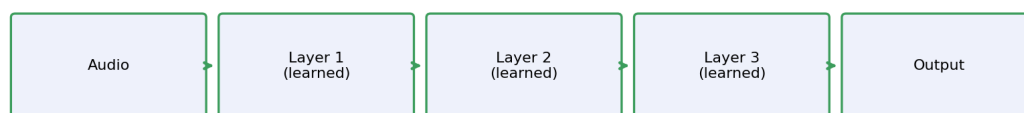


Fig. 1 — Classical hand-crafted pipeline (top) vs. end-to-end learned DNN pipeline (bottom). Compare with slides 4–5 (speech recognition, image classification).

1.1 Image Classification: from hand-crafted to fully learned

The lecture traces five points along a spectrum of image classifiers, from a single trainable classifier operating directly on raw pixels, to systems using hand-crafted gradients, histograms, and encoding schemes (k-means / sparse coding / Fisher Vectors / VLAD) before a trainable classifier, up to fully hand-crafted low/mid/high-level feature pyramids. The historical trend of deep learning is to push the **trainable/green boundary** further to the left, i.e., replace more and more hand-engineering with learning.

1.2 Benchmarks that drove progress

Public datasets provided a common yardstick to measure progress:

Dataset	#classes	Year	Notes
MNIST	10	1998	handwritten digits, grayscale
CIFAR-10	10	2009	32×32 colour images
CIFAR-100	100	2009	finer-grained categories
ImageNet	1000	2010	also includes localization & scene classification (ILSVRC)

On the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), the top-5 classification error fell from 28.2% (2010, shallow methods) to 3.57% (ResNet, 2015, 152 layers) — **below estimated**

human-level performance (~5%). Depth (number of layers) increased dramatically over this period: AlexNet (2012, 8 layers) → VGG/GoogLeNet (2014, 19–22 layers) → ResNet (2015, 152 layers). This was only possible after the introduction of residual/skip connections, which solved the vanishing-gradient/degradation problem of very deep plain networks (Russakovsky et al., IJCV 2015).

2. Images and Convolutional Neural Networks (CNNs)

2.1 Notation

The lecture fixes a consistent notation used throughout CNN discussions:

- **B**: batch size (number of samples processed together)
- **W, H**: width and height of an image or feature map
- **C**: number of feature channels = number of neurons per pixel; **C_{in}** for the current layer, **C_{out}** for the next layer. $C_{in} = 1$ for grayscale input, $C_{in} = 3$ for RGB colour input.
- **K × K**: convolutional filter kernel size (spatial extent of the filter)

2.2 VGG network as a running example

The VGG network (Visual Geometry Group, Oxford) illustrates the three fundamental layer types used in CNNs: **convolution** (+ReLU), **(max) pooling**, and **fully connected** layers, ending in a softmax over 1000 ImageNet classes. Spatial resolution shrinks ($224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$) while the channel depth grows ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), a very common CNN design pattern: trade spatial resolution for semantic depth.

2.3 Spatial extent — the key difference to MLPs

Unlike a fully connected (MLP) layer, where every output unit is connected to every input unit, a **convolutional layer has a spatial extent**: each output unit only looks at a small local neighbourhood ($K \times K$) of the input, and — crucially — the same filter weights are reused ("slid") across all spatial positions. This is called **weight sharing**.

Convolutional layer: same kernel weights re-used at every spatial position

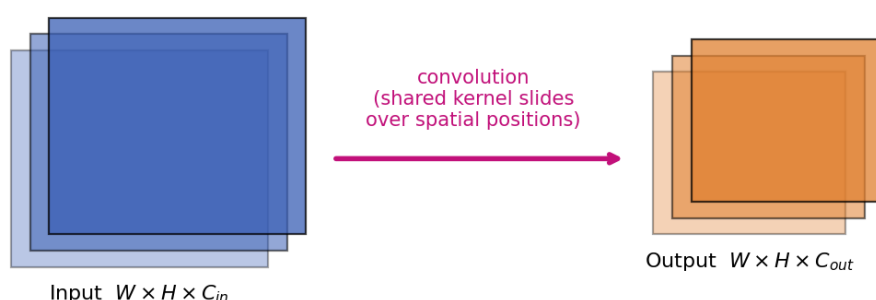


Fig. 2 — A convolutional layer maps an input volume ($W \times H \times C_{in}$) to an output volume ($W \times H \times C_{out}$) using a spatially-local, weight-shared filter (cf. slide 12).

2.4 Parameter counting: FC vs. Conv layer

This is one of the most frequently examined calculations. The two formulas from the slides:

Fully connected layer: $\#Weights = W \cdot H \cdot C_{out} \cdot (W \cdot H \cdot C_{in} + 1)$

Convolutional layer: $\#Weights = C_{out} \cdot (K \cdot K \cdot C_{in} + 1)$ ["weight sharing"]

The "+1" in both formulas accounts for the bias term per output unit / output channel. Note that the convolutional layer's parameter count is **independent of W and H** — this is exactly why CNNs generalize to different image sizes and scale to high-resolution images without an explosion of parameters.

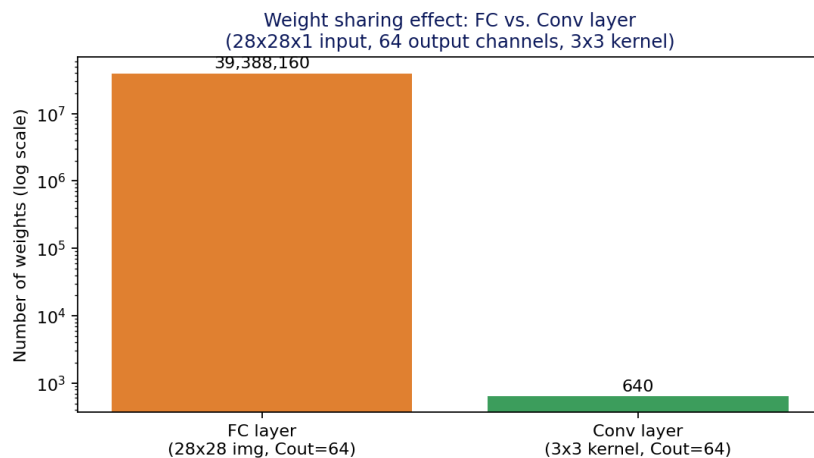


Fig. 3 — Worked numeric example: 28x28x1 input, 64 output channels, 3x3 kernel. FC layer needs ≈ 2.46 million weights; the equivalent conv layer needs only 640 weights (a reduction of more than 3 orders of magnitude).

Worked example (as in Fig. 3)

$W = H = 28$, $C_{in} = 1$, $C_{out} = 64$, $K = 3$:

- FC: $28 \cdot 28 \cdot 64 \cdot (28 \cdot 28 \cdot 1 + 1) = 50176 \cdot 785 = \mathbf{39,388,160}$ weights (displayed value in the chart uses $W \cdot H \cdot C_{out} \cdot (W \cdot H \cdot C_{in} + 1)$; recompute carefully on the exam!)
- Conv: $64 \cdot (3 \cdot 3 \cdot 1 + 1) = 64 \cdot 10 = \mathbf{640}$ weights

Exam tip: always double check whether the question wants weights only or weights + biases, and whether C_{in} refers to the current layer's input channels (as defined on slide 11).

2.5 Convolution operation — step by step

A convolution computes a weighted sum of a local input patch (highlighted red region) with the kernel, plus a bias b , sliding the kernel across the input to produce each output value.

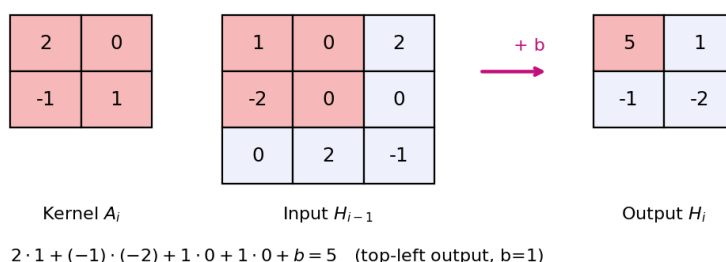


Fig. 4 — Numeric convolution example (slides 14–17): kernel $A_i = \begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}$, input $H_{(i-1)}$ 3x3, bias $b=+1$. Top-left output $= 2 \cdot 1 + (-1) \cdot (-2) + 1 \cdot 0 + 1 \cdot 0 + 1 = 5$.

Sliding the same 2x2 kernel over the 3x3 input (stride 1, no padding) produces a 2x2 output map: $H_i = \begin{bmatrix} 5 & 1 \\ -1 & -2 \end{bmatrix}$. Each output value reuses the identical 4 kernel weights + 1 bias — this is the essence of weight sharing.

2.6 Why convolutions work well for images

- **Local connectivity:** nearby pixels are more related than distant ones — a local receptive field is a sensible inductive bias.
- **Weight sharing / translation equivariance:** a cat detected in the top-left of an image should also be detected in the bottom-right using the same filter — this drastically reduces parameters and improves generalization / sample efficiency.
- **Hierarchical composition:** stacking conv layers lets the network build increasingly abstract features (edges → textures → parts → objects), matching the hand-crafted hierarchy of classical pipelines but learned.
- **Pooling** (e.g. max-pooling) adds a small amount of local translation invariance and reduces spatial resolution/computation going deeper into the network.

3. Sequences and Recurrent Neural Networks (RNNs)

3.1 Motivating applications

Many tasks require processing **sequences of variable length** rather than fixed-size vectors, where each time step depends on the sequence's history: multiple object recognition, image segmentation, object tracking, image generation, image annotation/captioning, machine translation, and text generation.

3.2 The recurrence idea

An RNN introduces a **feedback connection**: the hidden state H is updated from both the current input and the *previous* hidden state, using the **same shared weights and activation function** at every time step. This lets a single, small set of parameters process sequences of arbitrary length.

RNN unrolled over time: hidden state carries information forward (shared weights at every step)

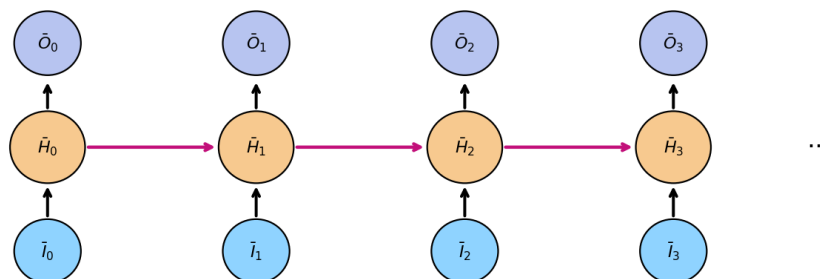


Fig. 5 — RNN unrolled over time (cf. slide 22): the hidden state H_t is a function of input I_t and previous hidden state $H_{(t-1)}$; weights are shared across all time steps t .

$$H_t = f(W_{ih} \cdot I_t + W_{hh} \cdot H_{(t-1)} + b), O_t = g(W_{ho} \cdot H_t + c)$$

Because H_t depends on $H_{(t-1)}$, which depends on $H_{(t-2)}$, and so on, the hidden state is, in principle, a function of **all previous inputs** — this gives RNNs "infinite" (unbounded) memory and lets them capture long-term dependencies, at least in theory.

3.3 Input/output mapping types

RNNs are flexible enough to support several input–output cardinalities:

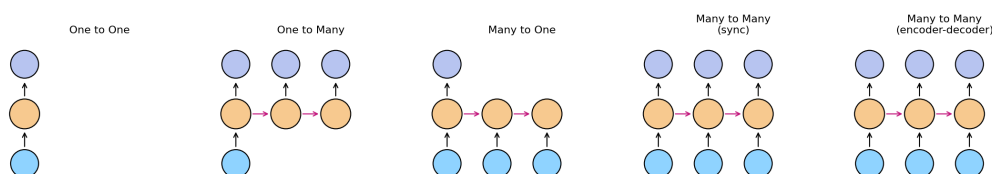


Fig. 6 — RNN mapping types (cf. slide 23).

Mapping	Example application
One to Many	Image captioning (image → sentence), music generation
Many to One	Action recognition (video → single action label)
Many to Many (aligned)	Object tracking (video → per-frame object location)

3.4 Beyond the slides: limitations of vanilla RNNs (research context)

Although not detailed in this lecture's slides, TU Dresden exams and follow-up research questions frequently probe the following well-known limitations, since they motivate later architectures:

- **Vanishing / exploding gradients:** backpropagation-through-time multiplies gradients across many time steps; repeated multiplication by the same recurrent weight matrix can make gradients shrink to (or blow up from) exponentially small/large values, making it hard to learn long-range dependencies in practice (Hochreiter 1991; Bengio et al. 1994).
- **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)** introduce gating mechanisms (input/forget/output gates) and an explicit cell state to control what information is kept, overwritten, or forgotten — mitigating the vanishing gradient problem and enabling longer effective memory.
- **Sequential computation:** because H_t depends on $H_{(t-1)}$, RNNs cannot be parallelized across the time dimension during training/inference — this is the key practical bottleneck that self-attention/Transformers solve (see Section 4).

4. Text Data, Self-Attention and Transformers

4.1 Representing words: one-hot vs. embeddings

Before feeding text into a network, words must be represented numerically. A dictionary D defines the vocabulary. **One-hot encoding** represents each word as a sparse, high-dimensional, hard-coded indicator vector. A **word embedding** (Bengio et al., 2003 — Neural Probabilistic Language Model) instead maps each word to a dense, lower-dimensional vector that is **learned from data**, such that semantically similar words end up close together in embedding space.

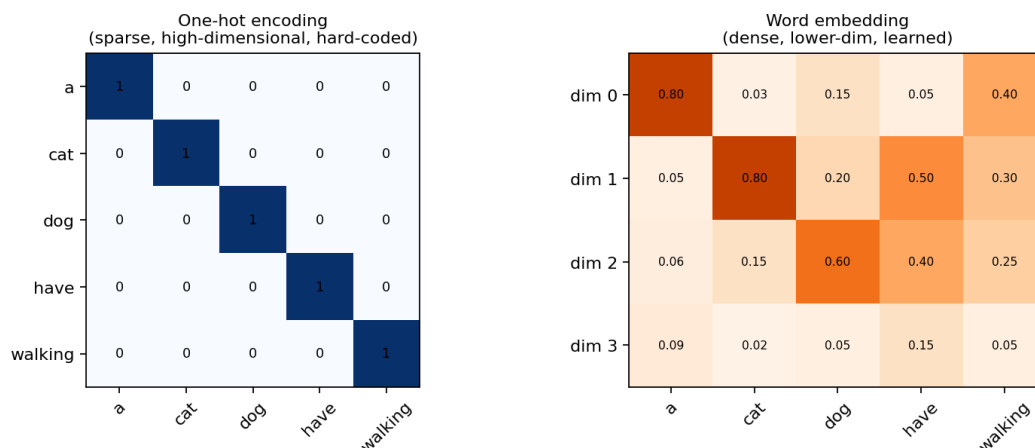


Fig. 7 — One-hot encoding (sparse/high-dim/hard-coded) vs. learned word embeddings (dense/low-dim/learned), cf. slide 25.

4.2 From RNNs to self-attention

RNNs process sequences strictly left-to-right (or right-to-left), so information from distant tokens must flow through many recurrent steps, making long-distance dependencies hard to capture and preventing parallel computation. **Self-attention** (Vaswani et al., "Attention Is All You Need", NeurIPS 2017) instead relates **all tokens of a sequence with each other directly** in a single layer, computing a new representation for each token as a weighted combination of all other tokens' representations.

Self-attention: "sat" attends to all tokens (line thickness = attention weight)

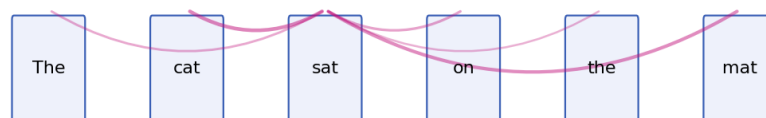


Fig. 8 — Self-attention: each token (e.g. "sat") attends to every other token in the sequence; line thickness indicates the learned attention weight.

Two crucial advantages over RNNs (slide 26):

- **Captures long-distance dependencies more easily**, since the "path length" between any two tokens is always 1 attention step, regardless of their distance in the sequence.

- **Calculation can be parallelized** across the sequence dimension (all token representations are computed simultaneously via matrix multiplications), unlike the inherently sequential recurrence of RNNs — this is the main reason Transformers scale so well on modern parallel hardware (GPUs/TPUs).

4.3 The Transformer architecture

The original Transformer is an **encoder-decoder** architecture. Each encoder block consists of a self-attention sub-layer followed by a position-wise feed-forward sub-layer, each wrapped with a residual "Add & Normalize" step. Each decoder block additionally contains an **encoder-decoder (cross) attention** sub-layer that lets the decoder attend to the encoder's output. Since attention itself has no notion of order, a **positional encoding** is added to the input embeddings to inject sequence-position information.

Transformer architecture (encoder-decoder)

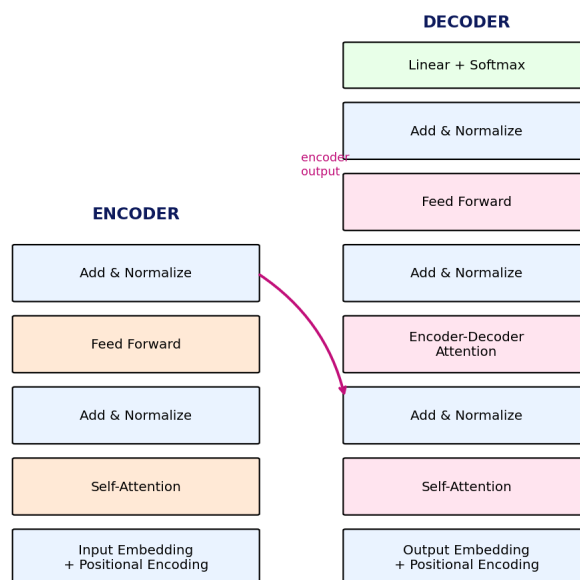


Fig. 9 — Simplified Transformer architecture (cf. slide 27; full illustration: jalammar.github.io/illustrated-transformer).

4.4 Model families and applications

Transformer model families and typical applications

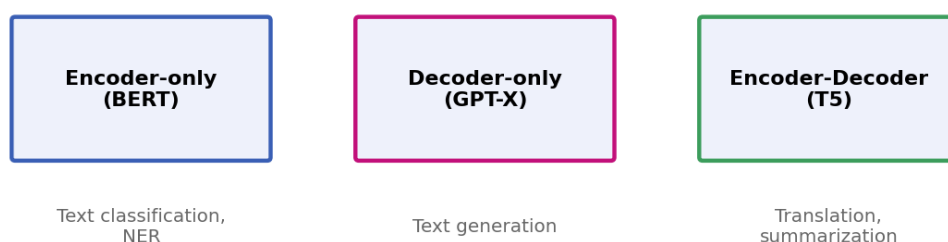


Fig. 10 — Encoder-only, decoder-only, and encoder-decoder Transformer families (cf. slide 29).

Family	Example	Typical use
Encoder-only	BERT (Devlin et al., NAACL 2019)	Text classification, named entity recognition — bidirectional context, no generation
Decoder-only	GPT-X (Radford & Sutskever, 2019)	Autoregressive text generation — causal (left-to-right) self-attention
Encoder-Decoder	T5 (Raffel et al., JMLR 2020)	Translation, summarization — frames every task as text-to-text

Reference for hands-on explanation and implementation: huggingface.co/learn/nlp-course/.

5. Cheat-Sheet: Key Formulas

Concept	Formula / Fact
FC layer weights	$W \cdot H \cdot C_{\text{out}} \cdot (W \cdot H \cdot C_{\text{in}} + 1)$
Conv layer weights	$C_{\text{out}} \cdot (K \cdot K \cdot C_{\text{in}} + 1)$
Conv output size (no pad, stride 1)	$(W - K + 1) \times (H - K + 1)$
Conv output size (general)	$\lfloor (W - K + 2P) / S \rfloor + 1$ (P = padding, S = stride)
RNN hidden update	$H_t = f(W_{\text{ih}} \cdot I_t + W_{\text{hh}} \cdot H_{(t-1)} + b)$
Self-attention (per head)	$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$
Transformer building block	Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ Feed-Forward $\rightarrow$ Add&Norm
Bias term	" +1 " inside parameter-count formulas = one bias per output unit/channel

Note: the exact attention-scaling formula ($\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$) is not shown on the lecture slides (which only describe self-attention conceptually) but is standard knowledge expected at MS level and is very commonly asked as a bonus/research question.

6. Exam-Style Q&A; (TU Dresden format)

Short, definition- and concept-level questions of the kind typically used in written exams / oral exams for this module.

Q: What is the key difference between a classical (hand-crafted) pattern recognition pipeline and a Deep Neural Network pipeline?

A: In a classical pipeline, the intermediate feature representations (e.g. sound features, phoneme features, gradients, histograms) are manually designed by domain experts and only the final classifier is trained. In a DNN, all layers of representation are learned automatically from data via end-to-end training, given sufficient labelled data and compute.

Q: Define B, W, H, C_in, C_out, and K in the CNN notation used in the lecture.

A: B = batch size; W, H = width/height of the image or feature map; C = number of feature channels (neurons per pixel), with C_in the channels entering a layer and C_out the channels it produces (C_in = 1 for grayscale, 3 for RGB inputs); K×K = the spatial size of the convolutional filter kernel.

Q: Why do convolutional layers have far fewer parameters than fully connected layers operating on the same input/output volumes?

A: Because of weight sharing: a convolutional layer reuses the same $K \times K \times C_{in}$ (+bias) kernel at every spatial position, so its parameter count, $C_{out} \cdot (K \cdot K \cdot C_{in} + 1)$, does not depend on the spatial size $W \times H$ at all. A fully connected layer instead needs a separate weight for every (input, output) pixel pair, giving $W \cdot H \cdot C_{out} \cdot (W \cdot H \cdot C_{in} + 1)$ parameters, which grows quadratically with image size.

Q: What does it mean that a convolutional layer has a "spatial extent"?

A: Each output unit is computed from only a local neighbourhood ($K \times K$ window) of the input feature map, rather than from the entire input as in a fully connected layer — this is what is meant by "spatial extent", in contrast to MLPs (slide 12).

Q: List three public benchmark datasets for image classification mentioned in the lecture, and state one difference between them.

A: MNIST (10 classes, 1998, handwritten digits), CIFAR-10 (10 classes, 2009, small colour images), CIFAR-100 (100 classes, 2009, finer-grained categories), and ImageNet (1000 classes, 2010, natural images, also used for object localization and scene classification). They differ mainly in number of classes, image resolution/complexity, and dataset size.

Q: What historical trend in the ImageNet Challenge (ILSVRC) is illustrated by AlexNet, VGG, GoogLeNet and ResNet?

A: Classification error decreased sharply (28.2% → 3.57% top-5 error) as network depth increased dramatically (8 → 19/22 → 152 layers) between 2010 and 2015, showing that deeper CNNs (enabled by innovations such as residual connections in ResNet) achieved better accuracy on the same 1000-class task.

Q: What is the core idea of a Recurrent Neural Network (RNN)?

A: An RNN maintains a hidden state that is updated at every time step using the current input and the previous hidden state, applying the same shared weights/update rule at every step. This allows processing input sequences of variable length, in contrast to networks that require fixed-size input vectors.

Q: Name the four RNN input–output mapping types discussed and give one application example for each.

A: One-to-Many: image captioning (image $\rightarrow$ sentence) or music generation. Many-to-One: action recognition (video $\rightarrow$ single label). Many-to-Many (synchronized): object tracking (video $\rightarrow$ per-frame location). Many-to-Many (encoder-decoder): machine translation (sentence $\rightarrow$ sentence of possibly different length).

Q: Why is an RNN's hidden state said to give it "infinite memory"?

A: Because H_t is computed recursively from $H_{(t-1)}$ (which itself depended on $H_{(t-2)}$, etc.), the current hidden state is, in principle, a function of the entire input history — not just a fixed window — giving the network the capacity to model long-term dependencies (in theory, though in practice vanishing gradients limit this).

Q: What problem does self-attention solve relative to RNNs, and how?

A: Self-attention relates every token in a sequence directly to every other token in a single layer (instead of propagating information step-by-step through a chain of hidden states), which (1) makes long-distance dependencies easier to capture, since the path length between any two tokens is constant ($=1$), and (2) allows the computation to be parallelized across the sequence, unlike the inherently sequential RNN recurrence.

Q: Why do word embeddings replace one-hot encodings in modern NLP models?

A: One-hot vectors are sparse, high-dimensional (size = vocabulary size) and hard-coded (fixed, with no notion of similarity between words). Word embeddings are dense, much lower-dimensional, and learned from data, so that semantically related words end up close together in the embedding space — a much more useful representation for downstream learning.

Q: What are the three main sub-layers of a Transformer encoder block?

A: (1) Self-attention, (2) a position-wise feed-forward network, each followed by (3) a residual connection and layer normalization ("Add & Normalize").

Q: What additional sub-layer does a Transformer decoder block have that an encoder block does not, and what is its purpose?

A: The decoder has an Encoder-Decoder (cross-)attention sub-layer, positioned between the decoder's own self-attention and its feed-forward sub-layer. It lets each decoder position attend over the encoder's output representations, so the decoder can condition its generation on the full input sequence.

Q: Why is positional encoding needed in a Transformer but not in an RNN?

A: Self-attention has no inherent notion of token order — it treats the input as a set, not a sequence — so positional information must be injected explicitly (e.g. via sinusoidal or learned positional encodings added to the input embeddings). An RNN, by contrast, processes tokens strictly in order, so position is implicit in the recurrence.

Q: Give one example each of an encoder-only, a decoder-only, and an encoder-decoder Transformer model, with their typical application.

A: Encoder-only: BERT — text classification / named entity recognition. Decoder-only: GPT-X — text generation. Encoder-Decoder: T5 — translation and other text-to-text tasks.

7. Calculation / Worked-Example Questions

These numeric/derivation questions are the type most likely to appear as exercise or exam problems, since they test whether the formulas from Section 5 can actually be applied.

Q: A convolutional layer receives a 32×32 RGB image ($C_{\text{in}} = 3$) and applies 16 filters of size 5×5 ($C_{\text{out}} = 16$). How many trainable weights (including biases) does this layer have? Does the answer depend on the 32×32 spatial size?

A: $\# \text{Weights} = C_{\text{out}} \cdot (K \cdot K \cdot C_{\text{in}} + 1) = 16 \cdot (5 \cdot 5 \cdot 3 + 1) = 16 \cdot 76 = \mathbf{1,216}$ weights. The answer does NOT depend on $W \times H$ (32×32) — this spatial independence is exactly the benefit of weight sharing.

Q: For the same setup ($32 \times 32 \times 3$ input, 16 output channels), how many weights would an equivalent fully connected layer producing a $32 \times 32 \times 16$ output need?

A: $\# \text{Weights} = W \cdot H \cdot C_{\text{out}} \cdot (W \cdot H \cdot C_{\text{in}} + 1) = 32 \cdot 32 \cdot 16 \cdot (32 \cdot 32 \cdot 3 + 1) = 16384 \cdot 3073 = \mathbf{50,348,032}$ weights — roughly $41,000\times$ more than the convolutional layer's 1,216 weights.

Q: Given a 2×2 kernel $A = \begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}$, bias $b = 1$, and a 3×3 input $H = \begin{bmatrix} 1 & 0 & 2 \\ -2 & 0 & 0 \\ 0 & 2 & -1 \end{bmatrix}$, compute the full 2×2 output feature map (stride 1, no padding).

A: Sliding the kernel across all 4 valid positions gives: top-left = $2 \cdot 1 + 0 \cdot 0 + (-1) \cdot (-2) + 1 \cdot 0 + 1 = 5$; top-right = $2 \cdot 0 + 0 \cdot 2 + (-1) \cdot 0 + 1 \cdot 0 + 1 = 1$; bottom-left = $2 \cdot (-2) + 0 \cdot 0 + (-1) \cdot 0 + 1 \cdot 2 + 1 = -1$; bottom-right = $2 \cdot 0 + 0 \cdot 0 + (-1) \cdot 2 + 1 \cdot (-1) + 1 = -2$. Output $H_i = \begin{bmatrix} 5 & 1 \\ -1 & -2 \end{bmatrix}$ (matches Fig. 4 / slides 14–17).

Q: What is the general formula for the output spatial size of a convolution given input size W , kernel size K , stride S and zero-padding P ? Apply it to $W=28$, $K=3$, $S=1$, $P=0$.

A: Output size = $\lfloor (W - K + 2P) / S \rfloor + 1$. For $W=28$, $K=3$, $S=1$, $P=0$: $(28 - 3 + 0) / 1 + 1 = \mathbf{26}$, giving a 26×26 output feature map.

Q: An RNN has an input dimension of 10, a hidden state dimension of 20, and an output dimension of 5. How many parameters (weights + biases) are in the input-to-hidden, hidden-to-hidden, and hidden-to-output weight matrices combined?

A: Input-to-hidden: $10 \cdot 20 = 200$ weights + 20 biases (biases can be counted once, shared with hidden-to-hidden). Hidden-to-hidden: $20 \cdot 20 = 400$ weights. Hidden-to-output: $20 \cdot 5 = 100$ weights + 5 biases. Total $\approx 200 + 400 + 100 + 20 + 5 = \mathbf{725}$ parameters (exact accounting of shared biases may vary by convention — state your assumptions clearly in an exam).

Q: If a Transformer processes a sequence of length n , what is the computational complexity of one self-attention layer with respect to n , and how does this compare to an RNN layer's complexity?

A: Self-attention computes pairwise interactions between all tokens: complexity is $O(n^2 \cdot d)$ per layer (d = representation dimension), since every token attends to every other token. An RNN layer is $O(n \cdot d^2)$ but must be processed sequentially over n steps (cannot be parallelized), whereas the Transformer's $O(n^2 \cdot d)$ work can be fully parallelized across n — this is why Transformers are more efficient in wall-clock time on parallel hardware for typical sequence lengths, even though their theoretical FLOP count can be higher for very long sequences (a key motivation for later efficient-attention research, see Section 8).

8. Research-Oriented Q&A; (beyond the slides)

These questions go past what is explicitly on the slides, connecting the lecture content to broader research context — useful for oral exams, seminar discussions, or deeper understanding. Where relevant, this is explicitly flagged as going beyond the lecture material.

Q: Why did ResNet succeed in training 152-layer networks when earlier plain (non-residual) CNNs degraded in accuracy beyond ~20 layers?

A: [Beyond slides] Very deep plain networks suffer from a degradation problem: adding more layers can make training error worse, not just overfitting, because gradients must flow through many stacked non-linear transformations. Residual (skip) connections let each block learn a residual mapping $F(x)$ added to an identity shortcut x , so gradients can flow directly through the identity path during backpropagation, and a block can default to the identity function if extra depth is not useful — making very deep networks at least as easy to optimize as their shallower counterparts (He et al., CVPR 2016).

Q: The lecture states self-attention "can more easily capture long-distance dependencies" than an RNN. What is the formal reason, in terms of path length between tokens?

A: [Beyond slides] In an RNN, information from token i must pass through $|i-j|$ recurrent steps to influence token j , so the effective "path length" for gradients/information grows linearly with sequence distance, exacerbating vanishing gradients for long sequences. In self-attention, any token can attend directly to any other token in a single layer, so the maximum path length between any two positions is $O(1)$, independent of sequence length — this is a key theoretical argument from Vaswani et al. (2017), Table 1.

Q: The quadratic $O(n^2)$ cost of self-attention is a well-known scalability bottleneck. Name two broad families of research approaches that address this, and briefly describe their idea.

A: [Beyond slides] (1) Sparse / local attention patterns (e.g. Longformer, BigBird) restrict each token to attend only to a subset of positions (local windows + a few global tokens), reducing complexity to near-linear. (2) Low-rank / kernel-based approximations (e.g. Linformer, Performer) approximate the softmax attention matrix with a lower-rank or kernelized form, avoiding the explicit $n \times n$ matrix, again yielding near-linear complexity. Both trade some modelling flexibility for scalability to much longer sequences.

Q: How does BERT's pre-training objective differ from GPT's, and how does this connect to their architectural difference (encoder-only vs. decoder-only)?

A: [Beyond slides] BERT is trained with a masked language modelling objective (predict randomly masked tokens using bidirectional context) plus next-sentence prediction, which requires access to context on both sides of a token — hence an encoder-only, bidirectional self-attention architecture (no causal masking). GPT is trained with a standard autoregressive (causal) language modelling objective — predict the next token given only previous tokens — which requires masking out future tokens in self-attention, matching a decoder-only architecture that naturally supports left-to-right text generation.

Q: T5 frames every NLP task as "text-to-text". What is the practical benefit of this unification, and what is the architectural consequence?

A: [Beyond slides] Casting classification, translation, summarization, question answering, etc. all as sequence-to-sequence text generation allows a single model, training procedure, and objective (loss function) to be reused across many tasks, simplifying multi-task training and transfer learning. Architecturally, this requires an encoder-decoder Transformer (as opposed to encoder-only or decoder-only), since the model must both understand an arbitrary input text (encoder) and generate

an arbitrary output text (decoder).

Q: Why is weight sharing in CNNs considered an "inductive bias", and what assumption about images does it encode?

A: [Beyond slides] An inductive bias is an assumption built into a model that helps it generalize from limited data. Weight sharing encodes translation equivariance: it assumes that a useful visual pattern (e.g. an edge or an eye) is equally likely to appear at any spatial position, so learning one filter and reusing it everywhere is a valid and much more sample-efficient strategy than learning a separate weight per pixel location (as an MLP would need to).

Q: Both CNNs (weight sharing over space) and RNNs (weight sharing over time) reuse parameters. What is the common statistical motivation behind both design choices?

A: [Beyond slides] Both exploit stationarity / repeated structure in the data domain: CNNs assume the statistics of natural images are (approximately) translation-invariant across spatial positions; RNNs assume the dynamics governing a sequence are (approximately) time-invariant across time steps. In both cases, reusing the same parameters everywhere the same underlying process/statistics apply reduces the number of free parameters, acting as a strong regularizer and improving sample efficiency relative to a fully unconstrained (fully connected) model.

Q: What limitation of the original Transformer's fixed-length positional encoding motivated later "relative" or "rotary" positional encoding schemes (e.g. RoPE)?

A: [Beyond slides] The original sinusoidal positional encodings are absolute (tied to a fixed position index) and were shown to generalize poorly to sequence lengths not seen during training. Relative positional encodings (encoding the offset between two tokens rather than their absolute positions) and rotary embeddings (which rotate query/key vectors in a way that naturally encodes relative position inside the dot product) improve extrapolation to longer contexts and are used in most modern large language models.

9. Likely Past-Exam-Style Questions & Model Answers

Modeled on the structure, phrasing, and depth typically expected in TU Dresden's "Digitization and Data Analytics" module exams (mixing short-answer, derivation, and applied/comparison questions). Use these to self-test; write your own answer first, then compare.

Q: (4 pts) Explain, in your own words, why deep learning is sometimes described as removing the need for "feature engineering". Use the speech-recognition example from the lecture in your answer.

A: Model answer: Traditional speech recognition pipelines required experts to hand-design a sequence of abstraction layers — sound features → phoneme features → language features → output — each requiring domain-specific engineering. A DNN instead learns this entire sequence of transformations directly from raw audio waveforms and labelled transcripts, discovering useful intermediate representations automatically through training, provided enough data is available. This removes the manual feature-engineering step and lets the same generic architecture/training procedure be reused across different domains (speech, images, text) simply by changing the training data.

Q: (6 pts) Derive the number of trainable parameters for a convolutional layer with input $64 \times 64 \times 3$, 32 output channels, and a 3×3 kernel. Then derive the number for a fully connected layer mapping the same input to a $64 \times 64 \times 32$ output. State the ratio between the two.

A: Model answer: Conv: $C_{\text{out}} \cdot (K^2 \cdot C_{\text{in}} + 1) = 32 \cdot (9 \cdot 3 + 1) = 32 \cdot 28 = 896$ parameters. FC: $W \cdot H \cdot C_{\text{out}} \cdot (W \cdot H \cdot C_{\text{in}} + 1) = 64 \cdot 64 \cdot 32 \cdot (64 \cdot 64 \cdot 3 + 1) = 131072 \cdot 12289 \approx 1,610,940,000$ (~1.61 billion) parameters. Ratio $\approx 1.61 \text{ billion} / 896 \approx 1.8 \text{ million}$ times more parameters for the FC layer — illustrating why FC layers are essentially never used directly on raw, high-resolution images.

Q: (3 pts) A colleague claims that RNNs cannot process inputs of different lengths, and that a fixed-size input vector is always required. Is this correct? Justify your answer.

A: Model answer: Incorrect. The defining feature of an RNN is that it applies the same update rule (shared weights) at every time step, which is exactly what allows it to be unrolled for as many steps as the input sequence requires — hence RNNs natively support variable-length sequences (e.g. sentences of different lengths), unlike standard feed-forward/MLP architectures, which need a fixed-size input vector.

Q: (5 pts) Compare RNNs and Transformers along two dimensions: (a) how they access earlier context, and (b) whether their computation can be parallelized over the sequence. Explain the consequence of each difference for training speed on modern GPUs.

A: Model answer: (a) RNNs access earlier context indirectly, via a hidden state that is updated step-by-step, so information from far-away tokens must pass through many intermediate steps. Transformers access all tokens directly through self-attention in a single layer, giving constant-length paths between any two positions. (b) RNN computation is inherently sequential (H_t requires H_{t-1}), preventing parallelization over the time dimension. Transformer self-attention computes all pairwise token interactions simultaneously via matrix multiplications, which parallelizes very efficiently on GPUs/TPUs. Consequence: for the same sequence length and model size, Transformers typically train substantially faster in wall-clock time than RNNs, which is a major practical reason for their adoption at scale.

Q: (4 pts) Why do BERT, GPT, and T5 use different subsets of the Transformer architecture (encoder-only, decoder-only, encoder-decoder) rather than all using the full encoder-decoder model?

A: Model answer: The architectural choice follows from the intended task. BERT is designed for understanding tasks (classification, NER) that benefit from bidirectional context over a fixed input — an encoder alone suffices and is more efficient than also carrying an unused decoder. GPT is designed for open-ended generation, which requires only autoregressive (causal), left-to-right prediction of the next token — a decoder alone suffices. T5 targets sequence-to-sequence tasks (translation, summarization) that genuinely need both a bidirectional understanding of an input sequence and an autoregressive generation of a (possibly different-length) output sequence — requiring the full encoder-decoder architecture with cross-attention.

Q: (3 pts) A student says: "Word embeddings are basically just a lookup table, so there is nothing 'learned' about them." Critically assess this statement.

A: Model answer: The statement is partially right and partially misleading. Structurally, a word embedding layer is implemented as a lookup table (an embedding matrix indexed by word ID). However, the *values* in that table are not fixed or hand-designed — they are trainable parameters, updated via backpropagation during training so that the resulting vectors capture useful semantic/syntactic regularities (e.g. words used in similar contexts end up nearby in embedding space). This is fundamentally different from a one-hot encoding, whose "lookup table" (the identity matrix) is fixed and carries no learned information.

Q: (6 pts) Sketch (in words) how you would extend the plain RNN discussed in the lecture to an LSTM, and explain what problem this extension solves. (Research-oriented / not explicitly on the slides.)

A: Model answer: An LSTM replaces the single hidden-state update of a plain RNN with an explicit cell state C_t plus three learned gates: a forget gate (decides what to discard from C_{t-1}), an input gate (decides what new information to write into C_t from the current input and previous hidden state), and an output gate (decides what part of C_t to expose as the hidden state H_t). Because the cell state is updated mostly additively (via the forget/input gates) rather than through repeated matrix multiplications, gradients can flow through many time steps largely unimpeded, mitigating the vanishing gradient problem that limits plain RNNs' ability to learn long-range dependencies.

10. Final Takeaways & Further Reading

- DNNs learn processing steps end-to-end that used to be hand-crafted.
- CNNs exploit weight sharing over space to drastically reduce parameters and gain translation equivariance — the standard choice for image data.
- RNNs exploit weight sharing over time to process variable-length sequences with a persistent hidden state — but suffer from vanishing gradients and sequential (non-parallel) computation.
- Self-attention/Transformers relate all sequence positions directly, giving constant path length between tokens and enabling full parallelization — now the state of the art for NLP (and increasingly vision/audio).
- Encoder-only (BERT), decoder-only (GPT-X), and encoder-decoder (T5) Transformer variants specialize the architecture to understanding, generation, and sequence-to-sequence tasks respectively.

Further reading (from the lecture references)

- Russakovsky et al.: *ImageNet Large Scale Visual Recognition Challenge*, IJCV, 2015.
- Bengio et al.: *A Neural Probabilistic Language Model*, JMLR, 2003.
- Vaswani et al.: *Attention Is All You Need*, NeurIPS, 2017.
- Devlin et al.: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL-HLT, 2019.
- Radford & Sutskever: *Improving Language Understanding by Generative Pre-Training*, 2018.
- Raffel et al.: *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, JMLR, 2020.
- Goodfellow, Bengio, Courville: *Deep Learning*, MIT Press, 2016.
- Andreas Geiger, University of Tübingen — Deep Learning Lecture (Lecture 7: CNNs, Lecture 8: Sequence Models).
- jalammar.github.io/illustrated-transformer/ — visual walkthrough of the Transformer.
- huggingface.co/learn/nlp-course/ — hands-on Transformer implementation course.
- d2l.ai — "Dive into Deep Learning", free interactive textbook.

End of study guide — based on TU Dresden CIDS/ZIH "Digitization and Data Analytics", Lecture 9: Deep Learning II.